GENESIS LUMINA • FIELD GUIDE NO. 01

FIRST EDITION • REMASTERED 2026

# ANALOG

## The Pre-Build System Design Guide for the Vibe Coding Era.

> "Everyone can build, but nobody can design. The real bottleneck of the AI era isn't writing code—it's knowing what to describe before you touch an LLM."

**Donovan Green**

Founder & Sovereign Operator • @investwithdg

SOVEREIGN

OPERATOR EDITION

EXECUTIVE ARCHITECTURE

# The Doctrine, Distilled

Something fundamental shifted in 2024. Cursor made coding conversational. Lovable turned ideas into interfaces in minutes. v0 generated production-ready components from descriptions. Claude started writing entire applications from prompts.

Suddenly, you didn't need to know how to code to build software. You just needed to know how to describe what you wanted. Millions of people who couldn't write a line of code six months ago are now shipping products, launching automations, and deploying systems.

> *"The real bottleneck wasn't technical skill. It was design clarity. And when you removed the technical friction, the design gap became impossible to ignore."*

**Analog** is the bridge between *"I know what I want to build"* and *"the AI can build it correctly."* It is the pre-build design layer that turns vibe coding from trial-and-error into precision engineering.

TABLE OF CONTENTS


**PART I: The Design Gap in the Vibe Coding Era**
The iteration trap, duct-tape codebases, and why developers used to ask questions.

**PART II: The PST Case Study (The Moment Everything Snapped)**
Three months into a $15k project, the closed MacBook, and the 72-hour paper turnaround.

**PART III: The Six Advantages of Paper**
Instant contradiction visibility, infinite visual canvas, and tactile neural encoding.

**PART IV: The 7-Step Analog Protocol**
Brain-dump, legend, cognitive colors, data movement, instinct redraw, and tool translation.

**PART V: From Paper to Perfect Prompts**
The 5-part prompt structure, voice-to-text workflows, and the transcription doctrine.

**APPENDIX: The Quick-Reference Desk Checklist**

The 7-step printable operational flight card.

PART I

# The Design Gap in the Vibe Coding Era

## The New Problem

Here's what the vibe coding era actually looks like for most builders:

You open Cursor. You have a clear idea (or so you think). You prompt: *"Build me a booking system with user accounts, calendar integration, payment processing, and email notifications."*

The AI builds something. It's... not right.

So you iterate: *"Actually, the admin needs to see all bookings, not just their own."* The AI rebuilds. Better. But still not right.

*"Wait, what happens when two people try to book the same slot at the same time?"* Another rebuild.

*"Hmm, I forgot the system needs to handle cancellations and refunds."* Another iteration.

*"Oh, and there should be automated reminders 24 hours before..."*

You're now on iteration seven. The codebase is held together with duct tape and good intentions. You've rebuilt core functionality three times. Edge cases keep emerging because you're discovering the system while you're building it.

> *"This isn't the AI's fault. Your prompts were clear. The code technically works. The problem is: the system was never designed. By you. It was bippity-boppity-booped into existence, one poop prompt at a time."*

## What Changed (And What Didn't)

**Before AI coding tools:** If you couldn't code, you hired a developer. The developer didn't just build. They asked questions:

- "What happens when a user tries to book outside business hours?"

- "Do admins have different permissions than regular users?"
- "Where does payment data get stored?"
- "What triggers the notification emails?"

Those questions forced you to think through the system. The developer was expensive ($5k–$50k for a project), but that expense came with built-in system design. They wouldn't start coding until the architecture made sense.

**With AI coding tools:** The AI doesn't push back. It doesn't ask clarifying questions. It doesn't force you to resolve contradictions. It just builds whatever you describe. If your description is incomplete, the system is incomplete. If your description is contradictory, the system has conflicts. If your description misses edge cases, the system breaks under real use.

*Garbage in, garbage out. Except now it happens in 30 seconds instead of 3 weeks.*

## The Iteration Trap

Most builders assume this is just part of the process now: *"Of course you have to iterate. That's how vibe coding works."* And yes, iteration is valuable. But there's a difference between:

| GOOD ITERATION | BAD ITERATION |
| --- | --- |
| **Product Development:** "The system works, but let me refine the UX based on user feedback." | **Trial-and-Error System Design:** "The system doesn't work because I didn't think through what happens when X and Y occur simultaneously." |

When you're designing through trial-and-error, you're not just moving slower. You're building on a foundation that was never solid. Every iteration adds complexity to a structure that was unclear from the start. Eventually, you hit the point where it's easier to rebuild from scratch than to keep patching.

## The Three Gaps Analog Fills

Analog synthesizes three disciplines that have never been packaged together for modern builders:

1. **Systems Engineering:** Components, data flows, states, transitions, dependencies, and failure modes. Analog gives you the thinking framework of a $150k+ Solutions Architect using nothing but paper and pen.

2. **Business Analysis:** Translating messy business requirements into clear specifications. Making implicit logic explicit before opening tools.
3. **The Vibe Coding Gap:** Turning paper architecture into AI-ready context so prompts produce systems that work on the first or second try.

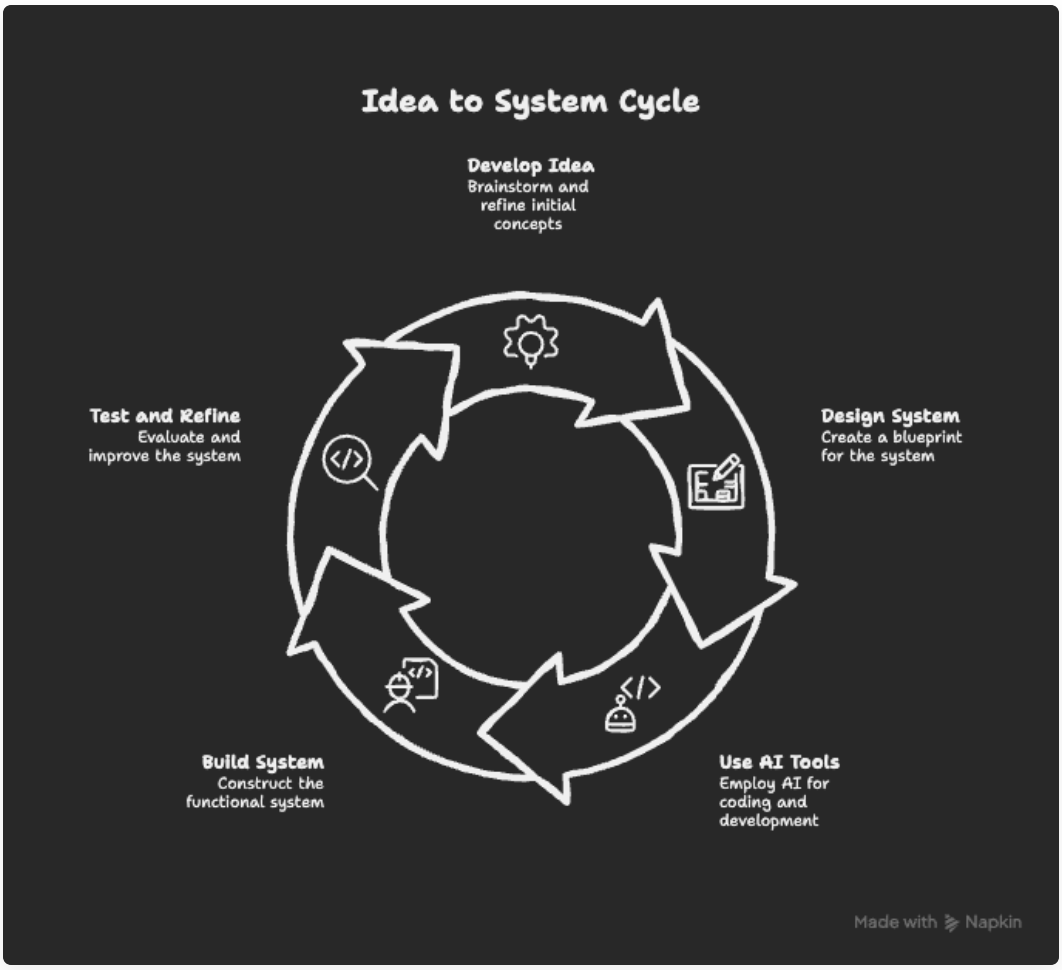



FIGURE 1.0: THE IDEA TO SYSTEM CYCLE — BRIDGING INTENT TO CODE

PART II

# The Moment Everything Snapped: The PST Story

---

CASE STUDY: THE DELIVERY TRANSPORT PLATFORM

3 MONTHS BLIND → 72 HOURS ON PAPER

### The Problem

I was three months into a $15k project that should've taken one. The client needed a delivery transport platform with four distinct user types: Admins, Dispatchers, Drivers, and Customers. The stack: Make.com for automation, HubSpot for CRM, Supabase for database, and Stripe for payments.

Each component made sense in isolation. But the whole system? It wasn't systeming. I kept discovering conflicts:

- Can dispatchers see customer data or not?
- What happens when a driver marks delivered but the customer disputes it?
- Should status updates trigger emails immediately or batch hourly?
- Who has permission to cancel a delivery once it's assigned?

Every day I was clicking through tabs: Make scenarios here, HubSpot workflows there, Supabase tables elsewhere. I could see individual pieces, but I couldn't see the macro-system.

## The Paper Moment

Finally, I stopped. Closed the MacBook. Grabbed my notebook, a black pen, and four highlighters:

- **Admin = Orange**
- **Dispatcher = Blue**
- **Driver = Yellow**
- **Customer = Pink**

I started drawing. Not code. Not database schemas. Not automation flows. Just the system as it actually was. Arrows showing data movement. Boxes showing decision points. Colors showing who sees what. Symbols showing where automations trigger.

Three hours later, I had the entire system on two sheets of paper. Front and back. More importantly: **I could see it all at once.**

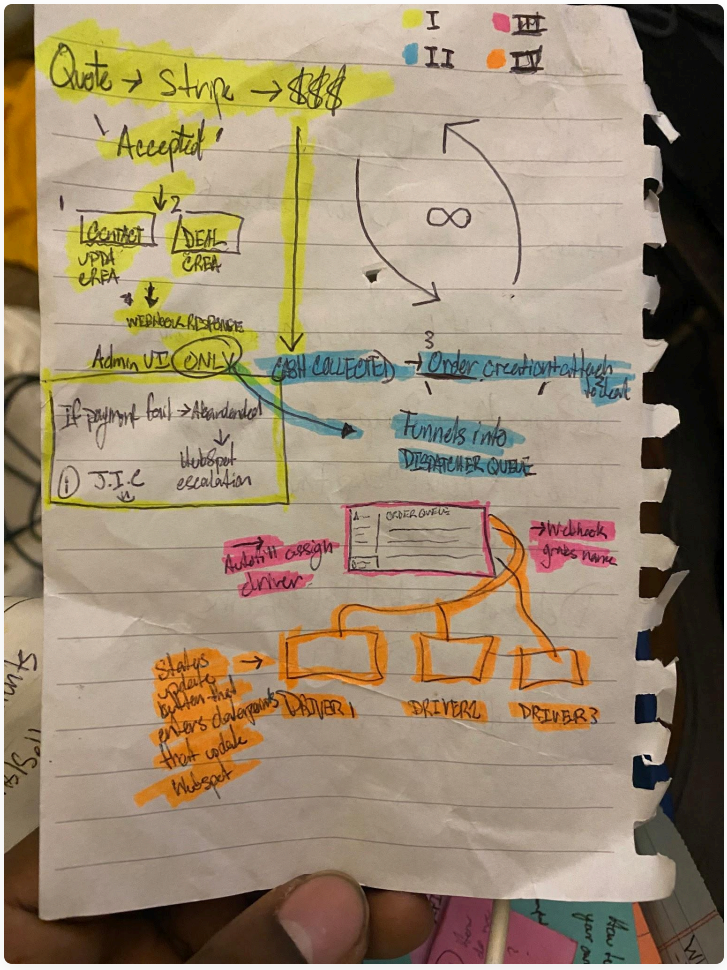



FIGURE 2.0: THE ORIGINAL PST ANALOG SPECIFICATION (ADMIN, DISPATCHER, DRIVER, CUSTOMER)

## What the Paper Revealed

The contradictions became impossible to ignore:

- Dispatchers needed read access to customer addresses, but the system was giving them full edit permissions (order pricing, etc.).

- Driver status updates were triggering three different email sequences (overkill).
- The "cancel delivery" action had different permission levels depending on timing, with zero logic to enforce it.
- Customer tracking pages were pulling data that didn't exist yet (a race condition waiting to happen).

None of this was obvious clicking through digital tabs. On paper? **Glaring.**

I trashed the whole repository. The paper map became my spec. From the start of the clean build to completion took me **72 hours**. Not because I coded faster (though shout out o3-pro and Claude Opus!), but because I wasn't coding blind anymore.

PART III

# The Six Advantages of Paper

The bottleneck in system design is the gap between the system in your head and the system in your tools. Your brain can hold a complex system for about 30 seconds before it starts dropping threads. Digital tools impose constraints—Miro forces box styles, Figma forces frames, Make forces modules. Paper has zero opinions.

> *"Research from the University of Waterloo found that students who drew concepts recalled nearly twice as much as those who only wrote notes. Drawing forces information processing in three simultaneous modes: visually, kinesthetically, and semantically."*

## The Six Structural Advantages:

1. **Instant Contradiction Visibility:** On a screen, two conflicting rules are buried in different tabs. On paper, they are six inches apart and impossible to ignore.
2. **Infinite Canvas for Early Thinking:** A single sheet of paper holds an entire system across 80% of your visual field. No zooming, no panning, no viewport limits. Shout out to Martin Heit (elite ball knowledge).
3. **Big Picture View Without Menus:** All actors, data flows, decision points, and edge cases are visible simultaneously. Patterns and anomalies emerge immediately.
4. **Separation of Unknowns, Decisions, and Flows:** Digital boxes look identical. On paper, your visual syntax separates known facts from question marks and automated triggers instantly.
5. **Mental Tempo That Exposes Edge Cases:** Drawing by hand is slower than typing. That cognitive pause is where edge cases surface: *"Does data really flow this way? What happens if B is offline?"*
6. **Tactile Learning That Embeds Architecture:** Constructing the system with your hands encodes it into muscle memory. Later, when prompting an AI, you are transcribing a system you already know intimately.

**[ CHECKPOINT: THE TRANSCRIPTION DOCTRINE ]**

Design is expensive (requires thinking). Transcription is cheap (just execution). Analog makes design cheap by doing it on paper, so the expensive digital tools become purely execution.

PART IV

# The 7-Step Analog Protocol

Read it once. Reference it forever. These seven steps transform system design from *"I hope this works"* to *"I know exactly what I'm building."*

### Step 1: Brain-Dump the System (15–30 min)

Get everything out of your head with zero organization, zero judgment, and zero optimization. Capture actors, triggers, data entities, processes, and edge cases. Quantity over quality.

> **[ CHECKPOINT 1 ]**
>
> Scan the page. Do you feel mentally spent? That is the signal that intake is complete.

### Step 2: Establish Your Legend

Create a visual language that separates different types of information. Keep your legend box in the corner of every page:

| SYMBOL | MEANING | OPERATIONAL EXAMPLE |
|---|---|---|
| ? | Unknown / Research Gap | "? How does payment gateway handle declined cards" |
| Δ / ∗ | Decision Point / Branch | "Δ If admin, show dashboard; if driver, show routes" |
| → | Transition / Flow | "Order submitted → Payment processed → Confirmed" |
| □ | Required Input | "□ User email address (validated)" |
| ☆ | Automation Trigger | "☆ Send confirmation email when status = confirmed" |
| ◆ | Human Action | "◆ Admin reviews and approves vendor application" |
| ⚠ | Failure Point | "⚠ Webhook arrives before database record exists" |

## Step 3: Assign Colors to Actors or States

Color is cognitive segmentation. Pick **ONE** dimension per map (User Roles OR Component Layers OR Data States). Never mix roles and states on the same sheet. Once Admin = Orange, Admin is always orange.

## Step 4: Draw How Data Moves (The Sacred Step)

Map every place data enters, transforms, forks, validates, and terminates. Every flow must have a terminus (stored, sent, displayed, or deleted). If an arrow doesn't end somewhere, you have a logic gap.

**THE 7 SACRED DATA QUESTIONS**

1. Where does this data come from?
2. What format is it in?
3. What happens to it?
4. Where does it go next?
5. What validates it?
6. What breaks if it's wrong?
7. What breaks if it doesn't arrive?

## Step 5: Redraw When Your Instinct Blinks

When the page looks tangled (more than 3 crossing arrows) or you're squeezing boxes into corners, **stop**. Do not fix it on the same page. Flip to a clean sheet and redraw from memory. First map = extraction. Second map = architecture.

## Step 6: Translate the Analog

Open your tools only when the map makes sense. Your map is your technical spec:

- Each arrow = One tool connection (Make / Zapier / n8n)
- Each entity = One database table (Supabase / Airtable)
- Each state = One status field or enum
- Each permission boundary = One auth check in code

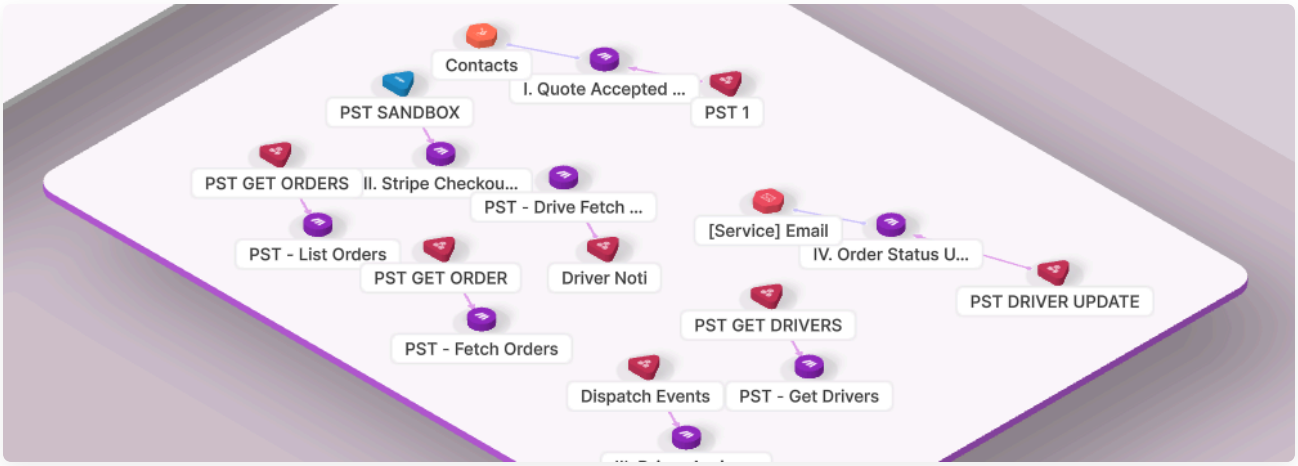



FIGURE 3.0: THE TRANSLATED DIGITAL WORKFLOW DERIVED DIRECTLY FROM THE PAPER MAP

## Step 7: Capture the Model for AI

Turn your paper map into structured context so AI coding tools build exactly what you designed on the first try.

PART V

# From Paper to Perfect Prompts

Most builders fail with AI because they think *"clear to me"* equals *"clear to AI."* It doesn't. AI needs explicit relationships and constraints. In order to drive the Ferrari, you have to know the track.

## The Five-Part Prompt Structure

1. **System Type & Actors:** Frame the system in one sentence: *"This is a delivery platform with 4 actors: admin, dispatcher, driver, customer."*
2. **Data Entry Points:** Specify triggers, input formats, and validation rules.
3. **Data Flow & Transformations:** Trace linear journeys: *"Form submission → Validation layer → Structured booking object → DB write → Trigger email."*
4. **Decision Points & Branching:** Explicitly call out every Δ: *"If date available, status='confirmed'. If date full, status='waitlist'."*
5. **Edge Cases & Failures:** List every ⚠ and ? marked on your paper map: *"What if the user submits twice in 30 seconds? Deduplicate by email+date."*

## The Structured Template Method

```
=== SYSTEM OVERVIEW ===
Type: [e.g. Booking platform, Dispatch automation]
Actors: [Admin, Dispatcher, Driver, Customer]

=== ACTORS & PERMISSIONS ===
[Actor 1]: Can [actions] | Sees [data] | Restricted from [actions]
[Actor 2]: Can [actions] | Sees [data] | Restricted from [actions]

=== DATA ENTRY POINTS ===
Entry Point 1: [Name]
- Trigger: [Webhook, Form submission, Cron]
- Required Fields: [Field name, Type, Validation rule]

=== DATA FLOWS ===
Flow 1: [Name]
Path: [Source] → [Validation] → [Transformation] → [Terminus]
Automation Trigger: [Event that fires action]

=== DECISION POINTS ===
Decision 1: At [Step]
If [Condition] → [Action A] | Else → [Action B]

=== EDGE CASES ===
- If [Failure Scenario] → [Expected Handling]
```

## The Build Workflow: Before vs. After Analog

| THE OLD WAY (DESIGN THROUGH TOOLS) | THE NEW WAY (DESIGN FIRST, BUILD SECOND) |
|---|---|
| **Day 1:** Open Cursor, write vague prompt, iterate 3x. | **Day 1 (AM):** Analog mapping (3 hours). Brain-dump, legend, colors, data flows, redraw. |
| **Day 2:** Discover edge cases, rebuild core logic, break yesterday's code. | **Day 1 (PM):** Translate map into AI context. AI builds first component correctly. |
| **Day 3:** Discover auth conflicts, rewrite permissions. | **Day 2:** Build remaining components. Testing confirms design. Zero surprises. |
| **Day 4–7:** Patchwork fixes. Unclear if all edge cases covered. Fragile build. | **Day 3:** Integration and polish. System shipped with full confidence. |
| **Time:** 7+ days of confused building. Quality: Fragile. | **Time:** 3 days of clean execution. Quality: Solid & Scalable. |

APPENDIX

# The Quick-Reference Desk Checklist

### □ STEP 1: BRAIN-DUMP (15–30 MIN)

- [ ] Set timer. Dump actors, triggers, data, and edge cases. Quantity over quality.
- [ ] Checkpoint: "I think that's everything I know right now."

### □ STEP 2: ESTABLISH LEGEND

- [ ] Draw legend box: ? (unknown), Δ (decision), → (flow), □ (input), ☆ (automation), ◆ (human).
- [ ] Checkpoint: "I can distinguish element types at a glance."

### □ STEP 3: ASSIGN COLORS

- [ ] Pick ONE dimension (Roles, Systems, or States). Assign highlighters. Stay consistent.
- [ ] Checkpoint: "I can instantly see which parts belong to which actor."

### □ STEP 4: MAP DATA FLOWS (THE SACRED STEP)

- [ ] Mark entry points (□), transformations (→), branches (Δ), triggers (☆), validations (✓).
- [ ] Ensure every single path has a terminus. Answer the 7 Sacred Data Questions.

### □ STEP 5: REDRAW IF NEEDED

- [ ] If more than 3 crossing arrows or page is cramped: flip sheet and redraw from memory.
- [ ] Checkpoint: "Someone unfamiliar can grasp the system in 5 seconds."

### ☐ STEP 6: TRANSLATE TO TOOLS

- [ ] Arrow = connection. Star = trigger. Triangle = router. Entity = DB table. Work linearly.
- [ ] Checkpoint: "I'm translating a solved problem, not exploring."

### ☐ STEP 7: CAPTURE FOR AI

- [ ] Voice walkthrough, photo annotation, or structured template. Include full system context.
- [ ] Checkpoint: "I am handing off a complete specification."

## Next Waypoint in the System

Now that your systems are designed on paper, how do you harvest breakthroughs from your AI conversations and turn them into compounding intellectual property?

**CONTINUE TO: THE HARVEST GUIDE ($97) →**